

Market-Based Task Allocation for Persistent Multi-Robot Rescue Exploration

Revised project proposal (working draft, v2) — supersedes the proposal of the initial submission

Halil Elbay, Paul Butter · Praktikum Verteilte Robotiksysteme, TU Darmstadt

11 August 2026

Contents

0. What changed since v1, at a glance	2
1. Motivation and problem statement	3
1.1 The extension that reframes the project	3
2. System setup	4
2.1 Platform and environment	4
2.2 Detection	4
3. Bidding	5
3.1 Corrected cost function	5
3.2 Why δ moved	5
4. Priority dynamics	6
4.1 Why this form	6
4.2 Consequence for the protocol	6
4.3 Base priorities per task type	6
4.4 Aging	7
4.5 Preemption cascades	7
5. Task types	8
6. Research objectives	9
7. Evaluation metrics	10
8. Methodology: three tiers	11
9. Scope and honest limitations	12
10. Expected results and open questions	13
11. Ecological and social considerations	14
12. Current status and next steps	15

This is an informal working document. It records where our thinking currently stands, including the parts we changed our minds about. It is deliberately more explicit about open questions and dead ends than the final report will be.

0. What changed since v1, at a glance

Aspect	v1	v2 (this document)
Bid convention	highest bid wins	lowest cost wins — the original sign structure inverted three of four terms
Priority p	scalar, announced with the RoI	function of waiting time , evaluated locally by each bidder
Weight δ	term $-\delta p$ in the bid	moved into a preemption criterion ; in the bid it was mathematically inert
Free parameters	$\alpha, \beta, \gamma, \delta$	$\alpha \equiv 1$ fixed; β, γ swept, δ swept separately
Exploration	separate behaviour	unified with allocation via a fourth task type, SURVEY
Task priorities	uniform	type-dependent base priority plus aging
Evaluation	single simulator	three tiers (fast abstract sim, Webots, real Pi-Pucks)
Framing	multi-robot task allocation	persistent surveillance with market-based allocation

Two of these were corrections of outright errors in v1 (rows 1 and 3). The rest are deliberate extensions.

1. Motivation and problem statement

Coordinating a team of autonomous robots that must jointly discover and service Regions of Interest (RoIs) raises a question with no obvious answer: which robot should take which task? Always sending the nearest robot ignores accumulated workload and remaining battery, and systematically overloads the same units until they fail first. Assigning at random wastes travel time, which in a rescue context is the scarcest resource of all.

Market-based allocation offers a middle ground. Robots bid for tasks from their own local state, the best-placed robots win, and no central coordinator makes decisions. This matters beyond elegance: in a disaster environment, a central allocator is a single point of failure, while radio links are intermittent because of concrete shadowing, multipath, and congested spectrum. An auction protocol degrades gracefully — it continues to operate within whatever subgroup is still connected.

1.1 The extension that reframes the project

Our initial proposal treated exploration and task allocation as two separate mechanisms: robots explore, and when something is found, an auction happens. On reflection this misses the more interesting problem.

A region that has not been observed for a long time is not “already searched”. It is *informationally stale*: a survivor may have moved, an aftershock may have destabilised a structure, a fire may have spread. In a real search operation, areas are revisited precisely for this reason.

We therefore treat **staleness itself as a task**. Cells of the arena that have not been observed for some time generate SURVEY tasks whose priority grows with the time since last observation. Exploration is no longer a separate behaviour but an emergent property of the same allocation mechanism.

This changes the framing of the work from multi-robot task allocation to **persistent surveillance with market-based allocation**, which we consider both a sharper problem statement and one that is easier to position against existing literature (idleness-based patrolling, persistent monitoring).

2. System setup

2.1 Platform and environment

A team of Pi-Puck robots (e-puck2 base with a Raspberry Pi extension, differential drive, IR proximity sensors, on-board camera, WiFi) operates in a flat rectangular arena. Robots communicate peer-to-peer over UDP. An overhead camera provides robot poses.

On the overhead camera. We want to be explicit about this, because it is easy to mistake for cheating. The camera stands in for the localisation infrastructure a real operation would have: GNSS in the open, or a reconnaissance UAV or UWB anchors inside a collapsed structure without satellite reception. The conceptual line we hold is:

Localisation may be centralised. Decision-making may not.

A GNSS outage degrades navigation accuracy; every robot keeps working, less precisely. The failure of a central *allocator* stops the entire fleet. Perception is therefore partly infrastructure-supported while allocation is strictly peer-to-peer, and the report will state this rather than claim end-to-end decentralisation.

2.2 Detection

RoI detection is on-board (colour blob detection on the robot camera), so that “a robot discovers something and announces it” is meaningful rather than a distribution of a known list. If on-board vision proves unreliable in the lab, our fallback is a *virtual detection sensor*: the overhead camera knows all objects but delivers a detection only to a robot within $r_{\text{det}} \approx 0.3$ m with appropriate heading. Behaviour and communication patterns are identical; only perception is emulated. We would disclose this in the report.

3. Bidding

3.1 Corrected cost function

The formula in v1 was stated as a bid to be maximised, which made greater distance, higher workload, and fuller battery all *advantageous*. It is a cost function. We invert the convention: **lowest bid wins**, consistent with the SSI auction literature.

$$c_i(\tau) = \alpha \cdot \hat{d}_i(\tau) + \beta \cdot \hat{w}_i + \gamma \cdot (1 - \hat{e}_i)$$

with $\hat{d}$ the distance normalised by the arena diagonal, $\hat{w}$ the workload normalised by a capacity of three concurrent tasks, and $\hat{e} \in [0, 1]$ the state of charge, so that $(1 - \hat{e})$ makes a depleted robot expensive. All three terms lie in $[0, 1]$.

Normalisation. Within a single auction, bids are only compared relatively, so a common scaling does not change the ranking. We fix $\alpha \equiv 1$. This reduces the search from four parameters to two, which is the difference between a grid that runs in half an hour and one that does not.

Winner determination. Bids sorted ascending, the n_{req} cheapest win, ties broken by lower robot ID so that runs stay reproducible.

3.2 Why δ moved

In v1, priority entered the bid as $-\delta p$. Since p is announced with the RoI and is identical for every bidder in that auction, the term shifts all bids by the same amount and cannot change the winner. Any parameter sweep over δ would have measured noise.

For priority to have any effect it must enter a comparison *across* auctions, i.e. preemption:

$$\delta \cdot (p_{\text{new}} - p_{\text{cur}}) - (c_i(\tau_{\text{new}}) - c_i(\tau_{\text{cur}})) > \theta_{\text{hyst}}$$

A robot switches only if the priority gain, weighted by δ , exceeds the cost increase by more than a hysteresis margin. The hysteresis has an operational counterpart: constant re-tasking costs more travel time than the priority gain returns.

4. Priority dynamics

The two extensions we want — stale areas becoming tasks, and task types carrying different intrinsic importance — collapse into one function family.

$$p(\Delta t) = p_0 + (1 - p_0) \left(1 - e^{-(\lambda \Delta t)^k}\right)$$

Role	p_0	Δt measured from	Meaning of λ
SURVEY cell	0	last observation of the cell	expected event rate per cell
RoI task	$p_{\text{base}}(\text{type})$	announcement	aging rate

4.1 Why this form

For $k = 1$ the expression is derived rather than chosen. If events occur in a cell as a Poisson process with rate λ , then $1 - e^{-\lambda \Delta t}$ is exactly the probability that at least one undiscovered event is present. The priority of a SURVEY cell therefore is the expected event density, and λ is a physically interpretable quantity rather than a tuning knob.

The exponent k encodes the memory of the process. $k = 1$ is memoryless — appropriate for random new findings. $k > 1$ gives a Weibull form in which the hazard rate itself grows with time — appropriate for progressive processes such as aftershock damage or fire spread. We intend to sweep $k \in \{1, 2\}$ as a third experimental axis.

Two properties matter structurally: p stays bounded in $[0, 1]$, which the preemption criterion requires, and p saturates, so a long-neglected corner cannot permanently starve everything else.

Choosing λ . Half-life $t_{1/2} = \ln 2 / \lambda$, set to roughly one third of the run duration. Much shorter and every cell is maximally urgent immediately; much longer and no cell ever becomes urgent.

4.2 Consequence for the protocol

Because p changes during a run, ANNOUNCE can no longer carry a scalar priority. Re-announcing periodically is not acceptable, since communication overhead is one of our metrics. Instead, **ANNOUNCE carries the parameters** $(p_0, \lambda, k, t_{\text{ref}})$ and every receiver evaluates the function locally at bid time. Cost: four floats per announcement, zero additional messages.

4.3 Base priorities per task type

Our first instinct was Push > Surround > Guard, on the grounds of effort and blocking. Thinking about it in rescue terms suggests a different order: Surround contains a hazard whose damage *grows* with time; Guard marks and monitors a located find until human responders arrive, which in real urban search and rescue is among the most valuable things a robot can do; Push clears access, important but rarely immediately life-critical.

Rationale	Push	Surround	Guard
Effort and blocking	0.8	0.5	0.3
Rescue value (SAR)	0.4	0.8	0.6

We will use the SAR ordering as the main configuration and run the effort ordering as a sensitivity check. In the fast simulator this costs one configuration line. We consider the *justification* more important than the specific numbers, and the report will present it as such. SURVEY gets $p_0 = 0$ and thus always starts below any freshly announced RoI task — until waiting time reverses that. This is intended.

4.4 Aging

A fixed ranking alone would starve Guard tasks whenever the fleet is busy. The term $(1 - p_0)(1 - e^{-(\lambda\Delta t)^k})$ prevents this: p_0 is the starting value, not the final one. After sufficient waiting, any task overtakes any freshly announced one. This is the same mechanism as aging in operating-system schedulers.

4.5 Preemption cascades

Dynamic priorities turn a former edge case into a regular occurrence. A high-priority Surround task with $n_{req} = 3$ pulls three robots out of running assignments, producing three releases and three re-auctions, which may in turn trigger further preemptions.

Our rule:

Preemption is permitted only from the NAVIGATE state, never from WAIT_PEERS or EXECUTE.

A robot leaving WAIT_PEERS wastes not only its own travel time but that of the $n_{req} - 1$ robots already waiting at the site. The damage is multiplied. Operationally this corresponds to not breaking up a team that has already assembled. We add a 15 s per-robot cooldown between preemptions and log the preemption count as a diagnostic.

5. Task types

Type	n_{req}	Description	Real-world analogue
PUSH	2-3	Approach from opposing sides, relocate object to a target zone	Debris blocking an access route
SURROUND	3	Encircle an object simultaneously to contain it	Cordoning a spreading hazard
GUARD	1	Hold position at the RoI for a defined interval	Marking and monitoring a located find
SURVEY	1	Visit a stale cell and refresh its observation time	Re-sweeping a search sector

SURVEY is treated as background work: it does **not** count toward $\hat{w}$ and is interruptible at any time without hysteresis. Without this exemption, robots would always be nominally busy, $\hat{w}$ would saturate at 1, and β would lose all discriminative power — which would quietly invalidate one of the two axes we set out to study.

The arena carries two grids: a fine one (5 cm) for the coverage metric only, and a coarse one (≈ 40 cm, about 25 cells) for SURVEY. Using the fine grid for surveys would create 1600 candidate tasks. At most the three most urgent cells are announced per step, with a per-cell cooldown.

6. Research objectives

The contribution is a comparison along three axes plus one robustness study.

Axis 1 — allocation strategy. Market-based auction vs. naïve greedy (nearest robot always wins) vs. random assignment. Greedy is run in two variants, with and without SURVEY tasks, since greedy has no native notion of persistent surveillance and comparing it only against the survey-enabled market would be unfair.

Axis 2 — bid weights. Distance-dominant, load-balanced (β elevated), and battery-aware (γ elevated) parameterisations, over a grid in (β, γ) with $\alpha \equiv 1$.

Axis 3 — priority dynamics. Memoryless ($k = 1$) vs. aging ($k = 2$) staleness growth; SAR vs. effort-based base priority ordering; δ controlling preemption aggressiveness.

Robustness study — model misspecification. In the fast simulator, events are actually injected at a true rate λ_{true} , while the rate used in bidding, λ_{model} , is configured independently. We measure how detection latency degrades when the estimate is wrong by a factor of three in either direction. We consider this the most realistic question in the project, since a real operation never knows λ .

7. Evaluation metrics

Metric	Description
Task completion time	From announcement to confirmed completion
Coverage rate	Fraction of arena observed per unit time
Load imbalance	Standard deviation of per-robot task counts
Communication overhead	UDP messages and bytes per completed task
Mean cell idleness	Average time cells spend unobserved
Maximum cell idleness	Worst case; exposes starvation that the mean hides
Detection latency	From event injection to discovery — the actual rescue-relevant quantity
Preemption count	Cascade diagnostic

A caveat we will state openly in the results: with SURVEY tasks, coverage rate stops being a side effect and becomes a directly optimised quantity. Comparisons that ignore this would flatter the market strategy.

8. Methodology: three tiers

Rather than choosing between a custom simulator and Webots, we use both, with the auction logic written once and shared unchanged across all three levels.

Tier	Tool	Purpose
0	Custom Python simulator	Thousands of runs for parameter search. Point robots, no physics — deliberately unrealistic, optimised for speed
1	Webots (E-puck2 + Pi-puck PROTO)	Validation with real dynamics, collisions, sensor noise
2	Real Pi-Pucks	Demonstration that the approach transfers to hardware

The architectural rule that makes this work: the allocation module has no dependency on simulator, backend, or hardware code. Only a thin backend interface (pose, battery, drive-to, broadcast, receive, detect, clock) is swapped. This guarantees that the code optimised in Tier 0 is bit-identical to the code running on hardware — without it, the simulation results would not transfer and the central claim of the report would collapse.

Calibration. Tier 0 uses a coarse travel-time model $t = a + d/v_{\text{eff}}$, with a and v_{eff} fitted against roughly 20 point-to-point runs in Webots. This is what makes Tier 0 timings defensible rather than invented.

Tier 0 uses the same message format and the same log schema as the other two levels, so all metrics are computed by a single analysis script from three data sources — and so cross-tier comparison is meaningful at all.

9. Scope and honest limitations

We would rather state these upfront than have them read as failures.

Push will not be demonstrated on hardware. Cooperative pushing with 7 cm robots without grippers requires force closure and relative localisation accuracy that we do not believe we can achieve reliably in the available time. Push is studied in Tiers 0 and 1; Guard and, conditionally, Surround run on hardware.

Persistent surveillance is a simulation result. In a 2×2 m arena with five-minute runs, no cell is ever unobserved long enough for staleness dynamics to be informative. Running it on hardware would measure noise and consume lab time needed for the strategy comparison. We will disable SURVEY or set λ very low for the lab runs and say why.

The battery term uses a virtual model. Real Pi-Puck battery readings are noisy and barely drift over a short run. We use a modelled state of charge (consumption proportional to wheel usage) on hardware as well, and declare it.

Detection may be emulated. See §2.2. If on-board vision underperforms in the pilot session, we switch to the virtual detection sensor and disclose it.

The fleet is small. With N robots and $n_{req} = 3$, the auction becomes close to trivial if N is not comfortably larger than n_{req} . Scalability beyond the physical fleet size is examined in simulation only.

10. Expected results and open questions

We expect the market-based approach to outperform greedy and random on load balance and on maximum cell idleness, at moderate communication overhead, and we expect its advantage on raw completion time to be smaller than intuition suggests — greedy is, after all, locally optimal on exactly that metric. We expect battery-aware weighting to extend fleet uptime in longer runs at the cost of somewhat longer completion times.

More interesting than confirming this is what we do not know:

1. **Does a dominant configuration exist, or only a Pareto front?** We expect a front. If so, the finding is that there is no “best” parameter set, only a trade-off that an operational command would set before deployment based on the scenario.
2. **Does the ranking of strategies survive the reality gap?** If the ordering changes between Tier 0, Webots, and hardware, that discrepancy is the most valuable result we could obtain, and the discussion will be built around it.
3. **How badly does a wrong λ hurt?** If performance is flat in λ_{model} , the approach is practically usable without good priors. If it is sharp, that is a real limitation worth reporting.
4. **Do preemption cascades actually materialise?** We have a rule intended to prevent them, and a metric to detect them. We do not yet know whether the rule is sufficient.

A null result on any of these is acceptable and will be reported as such. We would rather present a cleanly measured absence of effect with a convincing explanation than an effect we cannot defend.

11. Ecological and social considerations

Four points we intend to address rather than treat as a required paragraph.

Algorithmic prioritisation of people. The parameter p implicitly decides who is helped first. Who sets it, on what basis, and who is accountable for a mis-weighting? A system that derives priorities automatically from sensor data moves an ethical decision into software. The defensible position is an explicit separation: the machine proposes an ordering, a human is accountable for it.

Resource and energy footprint. A robot fleet consumes lithium, rare earths, and manufacturing energy, and becomes electronic waste within a few years. The honest accounting is that this is justified only if the fleet actually saves lives or keeps people out of lethal environments. Incidentally, the battery term in our cost function is also an efficiency term: spreading load extends device lifetime and reduces replacement.

Dual use of the infrastructure. An overhead system that localises robots localises people equally well. Reconnaissance and coordination infrastructure developed for rescue is usable for surveillance without technical modification.

Relationship to human responders. The strong case for rescue robotics is not cost saving but deployment where humans cannot or should not go — radiation, structural collapse, toxic atmospheres. An argument built on staff reduction is both weaker and more open to criticism.

12. Current status and next steps

As of 11 August 2026:

- Bid formulation and priority dynamics are settled (this document)
- Detailed implementation specification exists as a separate briefing document
- Tier 0 implementation begins next; Webots calibration follows
- First lab session scheduled as a pilot whose only purpose is risk elimination: verifying that a complete auction runs between two physical Pi-Pucks over UDP

Open decisions we would welcome input on:

1. Whether the SAR-based or effort-based priority ordering is the more defensible default
2. Whether the number of available robots supports $n_{req} = 3$ for Surround on hardware
3. Whether persistent surveillance is better presented as a core contribution or as an extension, given the five-page limit